PriFFT: Privacy-Preserving Federated Fine-Tuning of Large Language Models via Hybrid Secret Sharing

Zhichao You, Xuewen Dong, *Member, IEEE*, Ke Cheng, Xutong Mu, Jiaxuan Fu, Shiyang Ma, Qiang Qu, and Yulong Shen, *Senior Member, IEEE*

***Abstract*—Fine-tuning large language models (LLMs) raises privacy concerns due to the risk of exposing sensitive training data. Federated learning (FL) mitigates this risk by keeping training samples on local devices, while facing the following privacy problems. (i) Recent studies show that adversaries can still infer private information in FL. (ii) LLM parameters are shared publicly during federated fine-tuning, while developers are often reluctant to disclose these parameters. (iii) Existing works focus on secure inference but neglects privacy-preserving fine-tuning. Inspired by above problems, we propose PriFFT, a privacy-preserving federated fine-tuning mechanism, to protect both model parameters and users' privacy. Due to considerable LLM parameters, we present hybrid secret sharing combining arithmetic secret sharing (ASS) and function secret sharing (FSS) to build secure operations and implement secure neural network layers and activation functions for fine-tuning. We optimize several secure protocols based on FSS, including reciprocal calculation, tensor products, natural exponentiation, softmax, sigmoid, hyperbolic tangent, and dropout. The hybrid secret sharing enables PriFFT to apply our optimized FSS protocols while combining ASS protocols to support complex computation without extra communication. PriFFT reduces execution time and communication overhead in privacy-preserving fine-tuning up to 59.1% and 77.0% without accuracy drop compared to the existing secret sharing methods.**

***Index Terms*—Privacy-preservation, secret sharing, federated learning (FL), fine-tuning.**

Received 20 May 2025; revised 10 October 2025; accepted 26 January 2026. Date of publication 6 February 2026; date of current version 12 May 2026. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB3107500, in part by the National Natural Science Foundation of China under Grant 62220106004, Grant 62232013, Grant 62402358, Grant 92467201, Grant 62502363, in part by the Key R&D Program of Shandong Province of China under Grant 2023CXPT056, in part by the Young Talent Fund of Association for Science and Technology in Shaanxi, China under Grant 20240138, in part by the Open Topics from the Lion Rock Labs of Cyberspace Security under Grant LRL24004, in part by the Fundamental Research Funds for the Central Universities under Grant ZDRC2202, in part by the Xidian University Specially Funded Project for Interdisciplinary Exploration under Grant TZJHF202502, in part by the China Scholarship Council (CSC), and in part by the Double First-Class Overseas Research Project of Xidian University. *(Corresponding authors: Xuewen Dong, Ke Cheng.)*

Zhichao You, Ke Cheng, Xutong Mu, Jiaxuan Fu, and Yulong Shen are with the School of Computer Science & Technology, Xidian University, Xi'an 710126, China, and also with the Shaanxi Key Laboratory of Network and System Security, Xi'an 710126, China (e-mail: zcyou@stu.xidian.edu.cn; chengke@xidian.edu.cn; xtmu@stu.xidian.edu.cn; jiaxuanfu@stu.xidian.edu.cn; ylshen@mail.xidian.edu.cn).

Xuewen Dong and Shiyang Ma are with the School of Computer Science & Technology, Xidian university, Xi'an 710126, China, and also with the Shaanxi Key Laboratory of Blockchain and Secure Computing, Xi'an 710121, China (e-mail: xwdong@xidian.edu.cn; shiyangma@stu.xidian.edu.cn).

Qiang Qu is with the Shenzhen Institues of Advanced Technology, Chinese Academy of Sciences, Beijing 100045, China (e-mail: qiang@siat.ac.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2026.3661572, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3661572

I. Introduction

LARGE language models (LLMs), such as GPT-4 [1], Llama [2], and BERT [3], are pre-trained on language data to understand the structure, grammar, semantics, and more complex language patterns and concepts of language. Due to the advantages of the LLM parameter scale, LLMs are widely used in search engines, healthcare, finance, and other fields. Fine-tuning pre-trained LLMs according to downstream tasks allows LLMs to achieve higher accuracy and improved performance in specific domains. However, the training samples of downstream tasks contain sensitive information, causing privacy leakage concerns when fine-tuning LLMs in downstream tasks [4]. For example, fine-tuning LLMs for the medical domain involves direct access to patient disease information and medical records. Therefore, it is essential to prevent sensitive data leakage while enhancing model performance in fine-tuning LLMs.

Federated Learning (FL) [5], [6], [7] ensures the security of individual data during model training, in which models are trained locally on the data owner's device, while only sharing model parameters. The training data remains local, and the model developer never gets the original training data, which provides privacy protection for sensitive sample data. Therefore, fine-tuning LLMs with FL, i.e., federated fine-tuning, provides privacy preservation for training data in downstream tasks [8]. Fig. 1 presents the framework of fine-tuning LLMs with FL where training samples remain local and only model updates are exchanged in the training process. Federated fine-tuning freezes most of the model's parameters and fine-tunes LLMs by adjusting a few selected parameters, alleviating the issue of limited computational resources for clients in FL.

Motivations: Federated fine-tuning faces two security threats from privacy leakage of model updates and parameter exposure of trained models. (i) Although the server cannot directly access training samples, attacks against FL show that **the server can still infer training sample information through model updates, which is the first security threat.** For example, model inversion attacks [9] reconstruct training samples through model gradients uploaded by clients, presenting the privacy risks of uploading model updates generated by local samples. (ii) Granting **clients direct access to trained model parameters in FL raises concerns about intellectual property, which is the second**

1545-5971 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

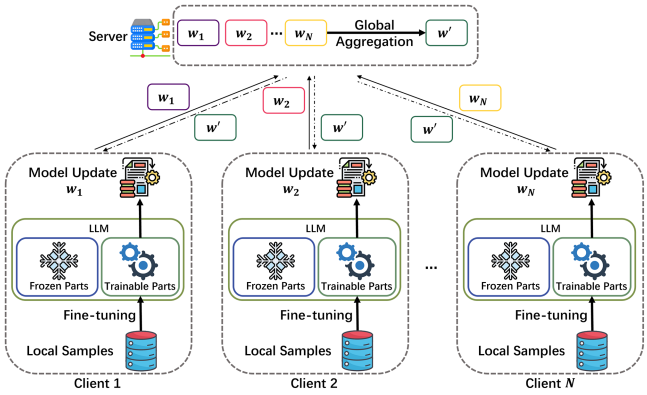


Fig. 1. A simple framework of federated fine-tuning.

security threat. Since model developers invest substantial computational resources, money, and effort in model training, they are cautious about disclosing trained model parameters to protect intellectual property and avoid commercial competition risks.

Challenges: Existing privacy-preserving FL mitigates the above privacy threats by implementing secure aggregation via secure multi-party computation (SMPC) [10]. The existing privacy-preserving mechanisms need to solve three challenges in federated fine-tuning of LLMs. Clients train the global model with local samples and generate model updates in plaintext. In the secure aggregation, the model updates are encrypted or perturbed before uploading. The server performs the model aggregation on the protected model updates with SMPC and generates updated global models. In the aggregation stage, the server cannot access the model updates, preventing the server from performing inference attacks on model updates and protecting clients' sample security. However, **the secure aggregation still requires model parameters to be exposed to clients, which cannot provide privacy preservation for model parameters, which is the first challenge**.

Existing privacy-preserving mechanisms for LLMs apply secret sharing during inference to protect both the model's parameters and the user's privacy [11], [12], [13]. However, these mechanisms **overlook privacy-preserving fine-tuning of LLMs and are not directly applicable to privacy-preserving federated fine-tuning**, as fine-tuning involves more complex secure computations compared to the inference, **which is the second challenge.** On the other hand, existing works rely on arithmetic secret sharing (ASS) [11] or function secret sharing (FSS) [13], **lacking a combined application of ASS and FSS, which is the third challenge.** ASS-based multi-party secure computation libraries (such as ABY2 [14] and CrypTen [11]) implement numerous foundational secure operations, thus eliminating the need to build privacy-preserving federated learning systems entirely from the ground up. In the meantime, FSS offers an alternative means of reducing the overhead in secure computation protocols. An advantageous strategy entails combining ASS and FSS into hybrid protocols with minimal additional overhead, thereby reducing resource consumption within the privacy-preserving system.

In this paper, we propose PriFFT, a privacy-preserving federated fine-tuning framework via hybrid secret sharing to protect both model updates and parameters. We propose efficient computation protocols based on FSS to decrease the communication overhead of bottleneck protocols in federated fine-tuning by minimizing the number of communication rounds and the volume of data exchanged. The hybrid secret sharing enables PriFFT to apply our optimized FSS protocols while combining ASS protocols to support complex computation without extra communication. Meanwhile, PriFFT leverages GPU acceleration to further reduce execution time. Compared to the fine-tuning under plaintext data, the evaluation shows that PriFFT causes a minor drop in model accuracy while providing privacy-preserving for both model parameters and updates. Meanwhile, PriFFT significantly reduces communication and computation overhead compared to implementations based on existing privacy-preserving mechanisms. The contributions of this paper are summarized as follows:

- *Privacy-preserving federated fine-tuning of LLMs:* PriFFT is the first privacy-preserving FL mechanism for fine-tuning pre-trained LLMs according to downstream tasks. PriFFT enables federated fine-tuning of LLMs while protecting local samples and model updates of clients, as well as the parameters of trained model.
- *Efficient implementation of bottleneck protocols:* Dealing with the substantial secure computations in fine-tuning, we propose efficient computation protocols, including reciprocal calculation, tensor product, natural exponential, softmax, hyperbolic tangent, and dropout functions based on FSS. The optimized protocols reduce execution time up to 62.5% and communication overhead up to 70.7% compared to existing secure computation protocols.
- *Fine-tuning based on hybrid secret sharing:* We present the hybrid secret sharing combining ASS and FSS to construct PriFFT. The hybrid secret sharing mechanism enables PriFFT to implement the proposed FSS protocols, reducing the bottleneck secure operation's overhead and facilitating complex secure computations based on ASS. The share conversion of hybrid secret sharing allows PriFFT to utilize both ASS and FSS protocols in privacy-preserving federated fine-tuning without incurring additional communication overhead.
- *Theoretical and experimental validity proof:* We ensure the security and efficiency of PriFFT through theoretical analysis. Evaluation results on real standard datasets demonstrate that PriFFT realizes comparable model accuracy to plaintext training while protecting both model parameters and updates. With the same model accuracy of the ABY2-based implementation, PriFFT reduces execution time and communication overhead in privacy-preserving fine-tuning up to 59.1% and 77.0% without accuracy drop compared to the existing secret sharing methods.

The rest of this paper is organized as follows. Section II summarizes the related work. Section III and Section IV provide the preliminaries and a system overview, respectively. Section V presents the design of PriFFT. Section VI discusses the complexity and provides the security proofs of PriFFT. Section VII evaluates PriFFT through comprehensive experiments. Finally, Section VIII concludes the paper.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

II. Related Work

A. Federated Fine-Tuning of LLMs

Several existing works propose FL mechanisms to fine-tune LLMs and protect clients' sample data. Qin et al. introduce zeroth-order optimization (ZOO) to realize full-parameter tuning of LLMs [15]. ZOO may struggle to achieve the same accuracy and efficiency as backpropagation when performing full-parameter fine-tuning of LLMs, since ZOO cannot precisely adjust model parameters by gradients. FedAdapter [16] implements adapter-based fine-tuning of LLMs with FL, which embeds adapters into LLMs and only optimizes adapter parameters according to downstream tasks. FedPETuning [17] develops a benchmark for four LLM fine-tuning methods, including adapter tuning, prefix tuning, and LoRA. FedPrompt [18] embeds soft prompts into global models and only adjusts the soft prompt parameters during the training process to realize communication-efficient fine-tuning of LLMs.

Exiting works on fine-tuning LLMs with FL do not provide privacy preservation for both model parameters and clients' model updates. As clients fine-tune models on local devices in FL, the server must distribute model parameters to clients, leading to the exposure of these trained parameters. Model developers are cautious about disclosing trained model parameters due to potential intellectual property issues and commercial competition. Besides, clients fine-tune models and generate model updates based on the sample data specific to downstream tasks. The server can directly access the model updates in model aggregation and reconstruct training samples through model inversion attacks [9] using model updates, presenting the privacy leakage risk when uploading model updates in plaintext.

B. Privacy-Preserving Federated Learning

Existing privacy-preserving FL mechanisms provide protection for model updates through different secure strategies, including differential privacy (DP), encrypted aggregation, and secret sharing. DP-based FL mechanisms protect model updates and parameters by adding noise to the sensitive data [19]. Clients perturb model updates with noise before uploading the model updates. Similarly, the server perturbs model parameters before distributing the parameters. Clients and the server receive perturbed sensitive data with noise, achieving protection for model updates and parameters. However, noise in perturbed updates and parameters brings model performance drops. When fine-tuning LLMs with extensive parameters, the perturbation noise in model updates and parameters significantly impacts model accuracy.

In encrypted aggregation, model updates are transmitted in ciphertext, preventing the server from accessing the sensitive data in plaintext. BatchCrypt [20] employs homomorphic encryption (HE) to implement secure aggregation, in which gradients are encrypted via HE and gradient aggregation is realized through secure computation. Tamer et al. present a privacy-preserving FL based on symmetric encryption, addressing the clients' dropout problem and enabling clients to validate the aggregated models [21]. SAFELearn [22] presents a communication-efficient and privacy-preserving FL framework that can be instantiated with SMPC or HE to defend against model inversion attacks on model updates. Abbas et al. present an HE-based privacy-preserving FL against model poisoning attacks with an internal auditor and Byzantine-tolerant gradient aggregation [23]. Few existing works of encryption-based FL consider the privacy leakage issues related to model parameters. When model parameters are encrypted, the models' local training must be done through secure computation. Fine-tuning LLMs with extensive encrypted model parameters imposes a significant computational load on clients.

The key idea of secret sharing-based FL mechanisms is to split model updates into multiple shares [24], [25]. Aggregation results are computed on shares, preventing the server from accessing clients' gradients during the aggregation. FastSecAgg [26] combines multi-secret sharing and fast Fourier transform to realize a trade-off between the number of secrets, privacy threshold, and dropout tolerance. ELSA [27] introduces two servers in FL and splits clients' gradients into two shares. Each server receives a share of clients' gradients and performs gradient aggregation on received shares. VerSA [28] utilizes key agreement protocols to generate seeds for pseudorandom number generators, which are then used to generate masks for model updates. The aggregated results on masked model updates can offset the random values of the masks. Additionally, the secret sharing mechanism in VerSA addresses the issue of client dropout. Secret sharing-based FL mechanisms for fine-tuning of LLMs present significant communication overhead challenges, as the non-linear computations in fine-tuning of LLMs require extensive communication between clients and the server. Considering the limited communication capabilities of clients' devices, clients may not be able to bear the communication consumption of FL mechanisms that directly apply secret sharing for privacy preservation.

Recently, Huang et al. combined TEE to realize privacy-preserving fine-tuning in which trainable parameters are put into the TEE during the fine-tuning process [29]. TEE guarantees the security of the proposed mechanism. However, the model parameters in TEE are plaintext. TEE is deemed not to provide provable security [30]. Therefore, the plaintext model parameters in TEE may confront privacy leakage.

The comparison of the existing works is briefly concluded in Table I. This paper presents PriFFT to protect model parameters and updates with provable security while reducing computation and communication overhead in federated fine-tuning of LLMs. Furthermore, to deal with the complex non-linear computations in fine-tuning, we propose several SMPC-friendly protocols based on FSS to significantly reduce communication overhead and execution time.

III. Preliminary

A. Federated Learning

In the training initialization phase, the server designs model structure f with initialized parameters ω_0, which are distributed to clients [5]. In the t-th training round, clients download the latest model parameters ω_t and update local models. Assume

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF RELATIVE WORKS

Mechanism	Federated Learning	Fine-tuning of LLMs	Sample Protection	Model Update Protection	Model Parameter Protection	SMPC Technique
ZOO [15]	√	√	√	×	×	×
FedAdapter [16]	√	√	√	×	×	×
FedPETuning [17]	√	√	√	×	×	×
FedPrompt [18]	√	√	√	×	×	×
Wei's work [19]	√	×	√	√	√	DP
BatchCrypt [20]	√	×	√	√	×	HE
LLAMA [13]	×	×	√	√	√	FSS
CrypTen [11]	×	×	√	√	√	ASS
SHAFT [12]	×	×	√	√	√	ASS
PriFFT (our solution)	√	√	√	√	√	ASS + FSS

that N clients participate in FL training, the local training data of client $i \in [1, \ldots, N]$ are samples x_i and labels y_i. The inference results are $f(x_i; w_t)$. Given the loss function L, the loss of client i in local training is $L(f(x_i; w_t), y_i)$, and gradients are $g_i = \partial L(f(x_i; w_t), y_i)/\partial \omega_t$.

Clients upload gradients g_i to the server in the aggregation phase of the t-th training round. The server averages gradients to get aggregated gradients as $\bar{g} = \sum_{i=1,\ldots,N} g_i$ and optimizes the parameters of the global models by aggregated gradients $\bar{g}$. The global model's updated parameters ω_{t+1} are distributed to clients for the subsequent round of local training. The training process is terminated once a specified number of training rounds has been completed.

B. Arithmetic Secret Sharing

We consider that secure computations are performed in the ring $\mathbb{Z}_{2^l}$, and all computations are modulo 2^l, where l is the bitwidth of the ring [14]. We map a real number $x_r \in \mathbb{R}$ to the item of the ring $x \in \mathbb{Z}_{2^l}$ by $x = \lfloor x_r \cdot 2^s \rfloor \bmod 2^l$ where s is a scale factor. The plaintext values are additively shared, and each party cannot learn information from the plaintext. Due to the space limitation, the sharing semantics and operations of ASS are given in the Appendix A.1, available online.

The existing ASS-based secure multi-party computing library CrypTen [11] implements numerous secure operations and constructs a comprehensive privacy-preserving machine learning training framework. We implement privacy-preserving federated fine-tuning for LLMs based on the existing ASS computation library, rather than developing all algorithms from scratch. Based on FSS, we propose optimizations for multiple calculations involved in fine-tuning to reduce the communication consumption for secure calculations in privacy-preserving fine-tuning. To enable direct integration of FSS-based protocols with existing multi-party secure computing libraries, we propose hybrid secret sharing, allowing seamless combination of both protocol types without additional communication costs.

C. Function Secret Sharing

Due to the improving communication efficiency, FSS [31] has been applied in various areas to implement privacy-preserving mechanisms [32], [33], [34]. We consider a 2-party FSS scheme in this paper. Given a function family $\mathcal{F}$, FSS splits a function $f : \mathbb{G}^{\text{in}} \to \mathbb{G}^{\text{out}}$ into two additive shares $\{f_0, f_1\}$ such that $f_0(x) + f_1(x) = f(x)$ where $x \in \mathbb{G}^{\text{in}}$ and $f(x) \in \mathbb{G}^{\text{out}}$. The security property of FSS requires that each share f_b hides f. The formal definition of FSS and the key generation in the offline stage are given in Appendix A.2, available online.

The evaluation $\text{Eval}(\sigma, k_\sigma, x)$ of FSS involves public input x, which is inconsistent in protecting sensitive data in privacy-preserving FL. Boyle et al. present secure evaluations in FSS based on offset functions [31]. The key idea is that for each gate $g : \mathbb{G}^{\text{in}} \to \mathbb{G}^{\text{out}}$ in the computation circuit, the input wire w_i and output wire w_j are masked by random offsets r_i and r_j. Specifically, offset functions $\hat{\mathcal{G}}$ includes functions of the form $g^{[r^{\text{in}}, r^{\text{out}}]}(x) = g(x - r^{\text{in}}) + r^{\text{out}}$. Each party evaluates FSS shares on the common masked input $w_i + r_i$ and obtains additive shares of the masked output $w_j + r_j$ to get the correct shares. The formal definition of FSS with the offset function is given in Appendix A.2, available online. In this paper, $\hat{x}$ refers to the fact that x is masked by an offset r, i.e., $\hat{x} = x + r$. The notation $\langle \hat{x} \rangle_b$ refers to an additive share of $\hat{x}$ held by party $b \in 0, 1$.

D. Share Conversion

The proposed privacy-preserving federated fine-tuning involves hybrid secure computing protocols based on ASS and FSS. The values are shared by ASS ($\langle x \rangle$) as discussed in Section III-B by default. When executing FSS-based protocols, the input of secure computing is $\langle \hat{x} \rangle$ which is then used to derive $\langle \hat{x} \rangle$. The conversion process is defined as follows:

Conversion from $\langle x \rangle$ to $\langle \hat{x} \rangle$ is implemented by having each party perform $\langle \hat{x} \rangle = \langle x \rangle + \langle r^{\text{in}} \rangle$. FSS-based protocols first perform the conversion to obtain $\langle \hat{x} \rangle$. Some FSS-based protocols invoke $\langle r^{\text{in}} \rangle$ to obtain computation results. To ensure that each party cannot infer information about x from $\langle \hat{x} \rangle_b$, we split r^{in} twice in the input conversion and offline stages.

Conversion from $\langle \hat{x} \rangle$ to $\langle x \rangle$ is required when an ASS-based protocol follows an FSS-based protocol. The conversion is implemented by omitting $\langle r^{\text{out}} \rangle$ in the output of an FSS-based protocol. Without $\langle r^{\text{out}} \rangle$, the output is converted to additive shares, while the key size is reduced l-bits due to the omission.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

IV. System Overview and Threat Model

A. Threat Model

Adversary's identity: We consider semi-honest servers and clients. The server and clients adhere to computation protocols while attempting to infer as much private information as possible from the calculation results. In FL [5], [6], [7], the server initiates the model training process, managing the collection of model parameters and the aggregation of updates contributed by users to produce updated model parameters. Therefore, the server possesses the complete plaintext parameters and aims to prevent clients from accessing the specific values of these parameters. Similarly, clients possess plaintext training samples and aim to prevent the server from gaining knowledge about these samples.

Adversary's goal: The server would infer sensitive information about the training samples through attacks against clients' model updates, such as model inversion, label inference, and membership inference attacks. Clients also infer model parameters during the fine-tuning process.

Adversary's ability and knowledge: The server possesses the model parameters from each training round, while clients retain local training data. Both the server and clients can directly access intermediate calculation results. In our work, both the server and the clients are assumed to be semi-honest, meaning they follow the protocol honestly but may attempt to infer additional information from the received data. However, in federated learning, there are also scenarios where malicious servers or clients may actively launch attacks. Our previous work has investigated methods for identifying malicious clients with gradient poisoning attacks in FL while preserving user privacy [35], [36]. Secure multi-party computation protocols that defend against malicious parties require a complete protocol redesign and incur substantial computational overhead. Preventing malicious servers or server dropouts requires redesigning the implementation framework by incorporating multi-server collaboration or decentralized technologies such as blockchain. Our work centers on supporting privacy-preserving fine-tuning of LLMs. The issue of malicious server behavior lies beyond the scope of this study and will be investigated in future research.

Defense goals: Since multiple attacks against FL can infer sensitive information from model updates, one of the defense goals is to protect uploaded model updates, i.e., the server cannot directly access the original model updates. Another defense goal is to guarantee that clients cannot obtain the model parameters by intermediate calculation results.

Party responsibilities: In this paper, we implement privacy-preserving federated fine-tuning of LLMs using secure multi-party computation based on secret sharing between the server and clients. In the common FL framework [5], [6], [7], it is standard for the server to hold plaintext model parameters while clients maintain plaintext training samples. Thus, the server and clients are required to participate in secure multi-party computation, each maintaining control over their own plaintext data. To protect model parameters and user gradients, the server and clients utilize secret sharing to encrypt plaintext data and mutually exchange the resulting shares. During the subsequent privacy-preserving fine-tuning of LLMs, both the server and clients perform secure computations on encrypted data, ensuring that neither party can access the true plaintext results.

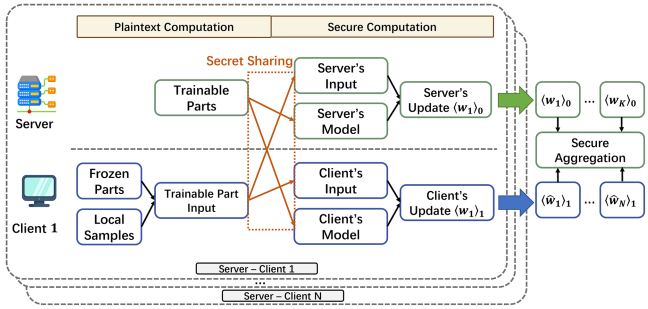


Fig. 2. A simple illustration of PriFFT framework.

B. System Model

Fig. 2 presents the framework of PriFFT, and we provide a more detailed implementation overview in Section V-A. There are N clients participating in FL training. For each client i, the training data is presented as (x_i, y_i), where x_i and y_i are training samples and labels. The model structure and parameters are f and ω, respectively. Parameters are divided into the frozen part $\omega_{\mathbf{F}}$ and the trainable part $\omega_{\mathbf{T}}$, where $\omega = \omega_{\mathbf{F}} \cup \omega_{\mathbf{T}}$. The frozen part $\omega_{\mathbf{F}}$ remains constant, and the trainable part $\omega_{\mathbf{T}}$ is adjusted by clients through local training.

The trainable model parameters $\omega_{\mathbf{T}}$ are transmitted to clients in the form of shares. The number of training rounds is T. In the t-th ($t \in [1, \ldots, T]$) training round, the server splits the latest trainable parameters $\omega_{\mathbf{T},t}$ into $\langle \omega_{\mathbf{T},t} \rangle_b$ ($b \in \{0, 1\}$) where $\omega_{\mathbf{T},t} = \langle \omega_{\mathbf{T},t} \rangle_0 + \langle \omega_{\mathbf{T},t} \rangle_1$. Similarly, client i randomly samples and splits a batch of sample (x_i, y_i) such that $x_i = \langle x_i \rangle_0 + \langle x_i \rangle_1$ and $y_i = \langle y_i \rangle_0 + \langle y_i \rangle_1$. The server and client i exchange shares of model parameters and sample data so that the server gets $\{\langle \omega_{\mathbf{T},t} \rangle_0, \langle x_i \rangle_0, \langle y_i \rangle_0\}$ and client i gets $\{\langle \omega_{\mathbf{T},t} \rangle_1, \langle x_i \rangle_1, \langle y_i \rangle_1\}$. Model parameters in the t-training round are $\omega_t = \omega_{\mathbf{F}} \cup \omega_{\mathbf{T},t}$. Since PriFFT splits $\omega_{\mathbf{T},t}$ into two shares, the protected model parameters are represented by $\hat{\omega}_{b,t} = \omega_{\mathbf{F}} \cup \langle \omega_{\mathbf{T},t} \rangle_b$ where $b \in \{0, 1\}$.

PriFFT transfers $f_{\mathbf{T}}$ to $\hat{f}_{\mathbf{T}}$ by our designed secure computation protocols such that $f(x_i; \omega_t) = \hat{f}(\langle x_i \rangle_0; \hat{\omega}_{0,t}) + \hat{f}(\langle x_i \rangle_1; \hat{\omega}_{1,t})$. Similarly, PriFFT transfers the loss function L to $\hat{L}$ and presents secure gradient computation method such that $L(f(x_i; w_t), y_i) = \hat{L}(\hat{f}(\langle x_i \rangle_0; \hat{\omega}_{0,t}), \langle y_i \rangle_0) + \hat{L}(\hat{f}(\langle x_i \rangle_1; \hat{\omega}_{1,t}), \langle y_i \rangle_1)$ and

$$\frac{\partial L(f(x_i; w_t), y_i)}{\partial \omega_{\mathbf{T},t}} = \sum_{b \in \{0,1\}} \frac{\hat{L}(\hat{f}(\langle x_i \rangle_b; \hat{\omega}_{b,t}), \langle y_i \rangle_b)}{\partial \langle \omega_{\mathbf{T},t} \rangle_b}. \quad (1)$$

We use $\langle g_{i,t} \rangle_b$ to represent the protected gradients of trainable parameters in the right term of (1). In other words, $\langle g_{i,t} \rangle_b$ is the share of the trainable parameter gradients generated by client i's samples in the t-th training round. Similarly, the left term of (1) is presented by $g_{i,t}$. In the aggregation phase of the t-th training

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

round, the server needs to calculate the aggregated gradients by

$$\bar{g}_t = \frac{1}{N}\sum_{i=1}^{N} g_{i,t} = \frac{1}{N}\sum_{i=1}^{N}\left(\langle g_{i,t}\rangle_0 + \langle g_{i,t}\rangle_1\right). \tag{2}$$

The server can directly get $g_{i,t}$ by $\sum_{b\in\{0,1\}}\langle g_{i,t}\rangle_b$ if client i uploads $\langle g_{i,t}\rangle_1$ to the server, leading to privacy leakage of clients' training samples due to reconstruction attacks against gradients. We apply the secure aggregation [28] to protect $\langle g_{i,t}\rangle_b$. Specifically, client i masks shared gradients $\langle g_{i,t}\rangle_1$ to double masked gradients $[\![g_{i,t}]\!]_1$ such that

$$\begin{cases} g_{i,t} \neq \langle g_{i,t}\rangle_0 + [\![g_{i,t}]\!]_1; \\ \bar{g}_t = \frac{1}{N}\sum_{i=1}^{N} g_{i,t} = \frac{1}{N}\sum_{i=1}^{N}\left(\langle g_{i,t}\rangle_0 + [\![g_{i,t}]\!]_1\right). \end{cases} \tag{3}$$

In other words, the server cannot restore $g_{i,t}$ by $[\![g_{i,t}]\!]_1$ but still get the correct aggregated gradients $\bar{g}_t$ by the sum of gradients from the server and clients.

The process of the t-th training round can be summarized as: **(i)** the server splits the trainable parameters $\omega_{\mathbf{T},t}$ into $\{\langle\omega_{\mathbf{T},t}\rangle_0, \langle\omega_{\mathbf{T},t}\rangle_1\}$ and sends $\langle\omega_{\mathbf{T},t}\rangle_1$ to client i; **(ii)** client i splits x_i and y_i into $\{\langle x_i\rangle_0, \langle x_i\rangle_1\}$ and $\{\langle y_i\rangle_0, \langle y_i\rangle_1\}$, and sends $\{\langle x_i\rangle_0, \langle y_i\rangle_0\}$ to the server; **(iii)** the server and client i takes the encrypted inputs and model parameters via presented secure computation protocols and generates encrypted model updates $\{\langle w_i\rangle_0, \langle w_i\rangle_1\}$; **(iv)** client i masks $\langle w_i\rangle_1$ to $[\![w_i]\!]_1$ and uploads $[\![w_i]\!]_1$ to the server; **(v)** the server calculates aggregated gradients $\bar{w}_t$ and optimizes the model parameters. The above process occurs in parallel between the server and each client.

V. PriFFT: Mechanism and Protocol Design

This section delves into the implementation details of PriFFT as well as the optimization for the related protocols used in PriFFT. Section V-A provides an implementation overview of PriFFT and demonstrates the application of the proposed protocols for constructing a privacy-preserving federated fine-tuning of LLMs from a high-level viewpoint. Section V-B presents the concept and implementation of the hybrid secret sharing. Section V-C presents the optimized computation protocols used in PriFFT. The detailed construction of the protocol includes numerous pseudo-code algorithms for both the offline and online phases.

A. Implementation Overview

Fig. 3 provides the design and implementation of PriFFT from a high-level perspective, which elaborates on and augments the framework presented in Fig. 2, illustrating the implementation of PriFFT via the proposed protocols. Fig. 2 aligns with the top process depicted in Fig. 3, where clients locally generate inputs for the trainable part models in plaintext and split these inputs through secret sharing. Before the training process or each training iteration, the model's trainable parameters are split in advance by secret sharing. Therefore, the training process that generates secure gradients used in updating parameters in fine-tuning is run in secret shares, which protects both the model parameters and clients' samples.

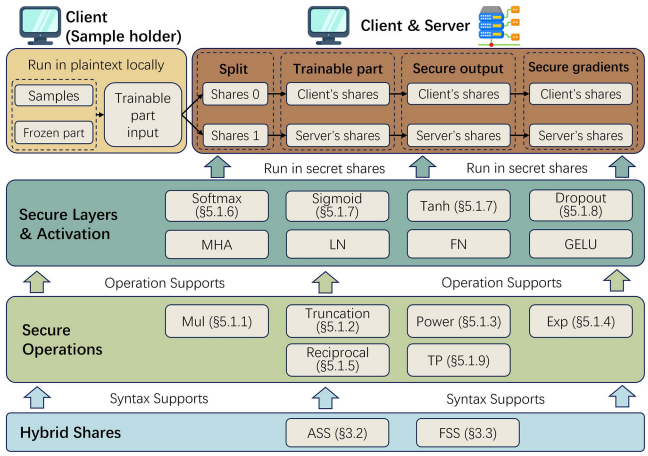


Fig. 3. The design and implementation of PriFFT from a high-level perspective.

PriFFT achieves privacy-preserving federated fine-tuning by integrating secure neural network layers and activation functions, illustrated as the second layer in Fig. 3. We optimize several secure activation functions like Softmax, Sigmoid, Tanh, along with dropout functions, whereas other secure model structures such as multi-head attention (MHA), layer normalization (LN), and feed-forward networks (FN) are executable through foundational secure protocols. The third layer in Fig. 3 represents the proposed secure operations, facilitating the execution of secure layers and activation functions. PriFFT optimizes secure operations, layers, and activation functions, resulting in reduced computing time and communication overhead when compared to existing secret sharing-based methods. The concept of hybrid shares proposed in Section V-B provides the syntax supports for higher-level protocols.

B. Hybrid Secret Sharing

We extend the concept of share conversion given in Section III-D to hybrid secret sharing, aiming to optimize secure computation protocols by minimizing overhead. The hybrid secret sharing is motivated by the following factors. While some protocols based on FSS (as mentioned in Sections V-C2-V-C7) are capable of minimizing computational overhead compared to ASS, there exist protocols that result in similar resource consumption (such as the multiplication). As a common secret-sharing technique, the ASS-based secret-sharing library currently available (like CrypTen [11]) executes numerous operations necessary for machine learning. Constructing a secure model's predictions and training system in machine learning via FSS entails reworking numerous foundational protocols and addressing the relevant engineering challenges. One more efficient approach involves crafting optimized protocols based on FSS utilizing hybrid shares to substitute the ASS protocols in the existing secret-sharing library, thereby minimizing the overhead of secure fine-tuning of LLMs.

We provide a brief illustration of the concept of hybrid shares using the examples presented in Fig. 4. Protocols 1 and 3 have inputs and outputs of identical types without requiring

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

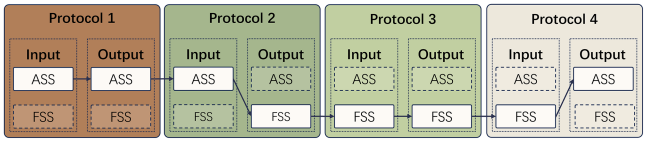


Fig. 4. Illustration of hybrid shares in combined protocols.

share conversion, and both are standard ASS or FSS protocols. During the implementation of protocol 2, if protocol 3 can be enhanced via FSS, then protocol 2 will produce FSS shares as its output. The aforementioned transformation can be carried out using $\langle \hat{x} \rangle = \langle x \rangle + \langle r^{\text{in}} \rangle$, where $\langle x \rangle$, $\langle \hat{x} \rangle$, and $\langle r^{\text{in}} \rangle$ represent, respectively, the ASS output before the share conversion, the FSS output after the share conversion, and the input randomness associated with FSS protocols (protocol 3). In a similar manner, if the subsequent protocol in protocol 4 receives ASS shares as input, protocol 4 is also capable of transforming FSS shares output into ASS shares. The implementation method is to omit $\langle r^{\text{out}} \rangle$ in the FSS output.

More intuitively, hybrid secret sharing combines ASS and FSS protocols and supports arbitrary combinations of the two to suit different computation tasks. Shares are classified into ASS and FSS types according to the splitting method. Only ASS shares can be input to the ASS protocol, and only FSS shares to the FSS protocol. In the absence of hybrid secret sharing, the ASS and FSS protocols produce only their respective types of shares, preventing interoperability between the two. Hybrid secret sharing provides the flexibility to convert output shares into the appropriate type required by the next protocol in the computation pipeline. Taking the example in Fig. 4, the output of protocol 2 can be converted into FSS shares, which match the input type required by protocol 3.

By integrating share conversion into secure computing protocols, hybrid shares can be incorporated into the mechanism to enable fully federated fine-tuning of LLMs. Because share conversion incurs minimal computational overhead and does not increase communication consumption, the FSS protocols can seamlessly substitute the ASS protocols within the current mechanisms.

C. Protocol Design

Based on the concept of hybrid shares, we introduce the optimized secure computing protocols developed by PriFFT in this section. Section V-C only presents the protocol design, while the detailed pseudo-code and further discussion are located in the appendix for interested readers due to space constraints.

1) Power Functions: Since several computations in PriFFT involve the secure square function, we present a secure square function based on offset functions of FSS to reduce the communication overhead. The straightforward way to implement the square function is using ASS multiplication given in Section III-B. The communication overhead of computing x^2 for a value $x \in \mathbb{Z}_{2^l}$ is $5l$ bits using ASS. We optimize the protocol based on FSS and reduce communication overhead from $5l$ bits to $3l$ bits. We leave the construction detail of the secure square function in Appendix B.3, available online. Similar to the construction of the secure square function, we present the secure multiplication based on FSS to implement multiplications with FSS, while the construction detail is given in Appendix B.1, available online. Besides, it is straightforward to construct the secure power function, which is similar to the construction of the secure square function. Due to the space limitation, the detailed construction of the secure power function is given in Appendix B.3, available online.

2) Natural Exponential Function: We consider iterative methods that approximate e^x with the linear computations. Our approximation comes from the limit definition that $e^x = \lim_{n\to\infty}(1 + x/n)^n$. We approximate the shares of e^x from the shares of x by $\langle e^x \rangle_b = (1 + \langle x \rangle_b / n)^n$. Assume that $n = 2^m$, we have

$$\left(1 + \frac{x}{n}\right)^n = \left(1 + \frac{x}{2^m}\right)^{2^m} = \left[\left(1 + \frac{x}{2^m}\right)^2\right]^{2^{m-1}}. \tag{4}$$

In other words, performing m squares for $(1 + x/2^m)$ and its subsequent results yields the result of $(1 + x/n)^n$ where $n = 2^m$. For a share $\langle x \rangle_b \in \mathbb{Z}_{2^l}$ of x, the computation in Equation (4) involves a truncation to compute $(1 + x/2^m)$ and m secure square functions. We define the natural exponential function gate $\mathcal{G}_{\text{exp}}$ as the family of functions $g_{\text{exp},l} : \mathbb{Z}_{2^l} \to \mathbb{Z}_{2^l}$ with input group $\mathbb{G}^{\text{in}} = \mathbb{Z}_{2^l}$, output group $\mathbb{G}^{\text{out}} = \mathbb{Z}_{2^l}$, and $g_{\text{exp},l}(x) := e^x$. Using (4) to approximate e^x, we denote the natural exponential function gate with FSS by $\hat{\mathcal{G}}_{\text{exp}}$ and the offset functions by:

$$\begin{aligned} \hat{g}_{\text{exp},l}^{[r^{\text{in}}, r^{\text{out}}]}(\hat{x}) &= g_{\text{exp},l}(\hat{x} - r^{\text{in}}) + r^{\text{out}} \mod 2^l \\ &= \left(1 + \frac{\hat{x} - r^{\text{in}}}{2^{m_{\text{exp}}}}\right)^{2^{m_{\text{exp}}}} + r^{\text{out}} \mod 2^l, \end{aligned} \tag{5}$$

where m_{exp} is the number of approximation iterations. The detailed construction of the offline stage $\text{Gen}_l^{\text{exp}}(r^{\text{in}}, r^{\text{out}}, \mathbf{r})$ and the online stage $\text{Eval}_l^{\text{power}}(b, k_b, \langle \hat{x} \rangle_b)$ is given in Appendix B.4, available online. The proposed protocol reduces the communication overhead of the secure exp in the online stage from $(l + 5lm_{\text{exp}})$ bits down to $(3l + 3lm_{\text{exp}})$ bits compared to the implementation of CrypTen [11].

3) Reciprocal Operation: We employ the Newton-Raphson method to approximate the reciprocal of the input. Given the function $f(y) = 1/y - x$, the iteration formula $y_{n+1} = y_n - f(y_n)/f'(y_n) = y_n \cdot (2 - xy_n)$ serves as a method to approximate $1/x$ as the number of iterations n becomes sufficiently large. We define the reciprocal gate $\mathcal{G}_{\text{recip}}$ as the family of functions $g_{\text{recip},l} : \mathbb{Z}_{2^l} \to \mathbb{Z}_{2^l}$ with input group $\mathbb{G}^{\text{in}} = \mathbb{Z}_{2^l}$, output group $\mathbb{G}^{\text{out}} = \mathbb{Z}_{2^l}$, and $g_{\text{recip},l}(x) := 1/x$. Using Newton's method to approximate $1/x$, we denote the reciprocal gate with FSS by $\hat{\mathcal{G}}_{\text{recip}}$ and the offset functions by:

$$\begin{aligned} \hat{g}_{\text{recip},l}^{[r^{\text{in}}, r^{\text{out}}]}(\hat{x}) &= g_{\text{recip},l}(\hat{x} - r^{\text{in}}) + r^{\text{out}} \mod 2^l \\ &= h_m + r^{\text{out}} \mod 2^l \end{aligned} \tag{6}$$

where

$$h_i = \begin{cases} y_0 \cdot \left[2 - (\hat{x} - r^{\text{in}}) \cdot x_0'\right] \mod 2^l, & \text{if } i = 0; \\ h_{i-1} \cdot \left[2 - (\hat{x} - r^{(i)}) \cdot h_{i-1}\right] \mod 2^l, & \text{else}, \end{cases} \tag{7}$$

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

y_0 is the initial value, m is the iteration round, and $r^{(i)}$ is the intermediate mask. In alignment with CrypTen [11], we use the default initial value $y_0 = 3e^{1-2x} + 0.003$ when y_0 is unspecified. Therefore, the communication overhead of the secure reciprocal operation depends on the provision of an initial value, as this determines the necessity of invoking the secure e^x function to compute the initial value. To minimize computational overhead when the range of x is relatively small, providing an initial value is beneficial. The detailed construction of the offline stage $\text{Gen}_l^{\text{recip}}(r^{\text{in}}, r^{\text{out}}, \mathbf{r})$ and the online stage $\text{Eval}_l^{\text{recip}}(b, k_b, \langle \hat{x} \rangle_b)$ is given in Appendix B.5, available online. The proposed protocol reduces the communication overhead of the secure reciprocal operation in the online stage from $(l + 5lm_{\text{exp}} + 10lm_{\text{recip}})$ bits down to $(5l + 3lm_{\text{exp}} + 6lm_{\text{recip}})$ bits compared to the implementation of CrypTen [11].

4) Softmax Function: The Softmax function takes a vector $\mathbf{x} = \{x_1, x_2, \ldots, x_K\}$ as input and normalizes $\mathbf{z}$ into a probability distribution consisting of K probabilities, i.e., the softmax function rescales $\mathbf{x}$ so that the output lies in the range $[0, 1]$ and sums to 1. The softmax function is given by $\text{Softmax}(\mathbf{x})_i = e^{x_i} / \sum_{j=1}^{K} e^{x_j}$. SHAFT [12] introduces an iterative technique for approximating the softmax function, bypassing the need to compute the input's maximum value in CrypTen. We decrease the cost of implementing the secure softmax function by improving the iterative approach utilized in SHAFT. Specifically, given a vector $\mathbf{x}$, the initial value is set to $y_0 = 1/K$. The protocol iteratively updates over m iterations as follows:

$$y_n = y_{n-1} + \frac{1}{m} \cdot (x - \langle x, y_{n-1} \rangle) \cdot y_{n-1}, \tag{8}$$

where $\langle x, y_{n-1} \rangle$ is an inner product calculation. We denote the softmax gate with FSS by $\hat{\mathcal{G}}_{\text{softmax}}$ and the offset functions by:

$$\begin{aligned} \hat{g}_{\text{softmax},l}^{[r^{\text{in}}, r^{\text{out}}]}(\hat{x}) &= g_{\text{softmax},l}(\hat{x} - r^{\text{in}}) + r^{\text{out}} \mod 2^l \\ &= h_m + r^{\text{out}} \mod 2^l \end{aligned} \tag{9}$$

where

$$h_i = h_{i-1} + \frac{1}{m} \cdot \left[\hat{x} - r^{(i)} - \left\langle (\hat{x} - r^{(i)}) \cdot h_{i-1} \right\rangle \right] \mod 2^l. \tag{10}$$

The detailed construction of the offline stage $\text{Gen}_{K \times l}^{\text{softmax}}(\mathbf{r}^{\text{in}}, \mathbf{r}^{\text{out}}, \mathbf{r})$ and the online stage $\text{Eval}_{K \times l}^{\text{softmax}}(b, k_b, \langle \hat{\mathbf{x}} \rangle_b)$ is given in Appendix B.4, available online. The proposed protocol reduces the communication overhead of the secure Softmax in the online stage from $(10l + 10lm_{\text{Softmax}})$ bits down to $(12l + 6lm_{\text{Softmax}})$ bits compared to the implementation of SHAFT [12].

5) Sigmoid Function and Hyperbolic Tangent: The sigmoid function is a common activation function, which is given by $\sigma(x) = 1/1 + e^{-1}$. The hyperbolic tangent function is a popular activation function in neural networks, which is given by $\text{Tanh}(x) = (e^{2x} - 1)/(e^{2x} + 1)$. The hyperbolic tangent, $\text{Tanh}(x)$, can be derived using the sigmoid function, $\sigma(x)$, as follows: $\text{Tanh}(x) = 2\sigma(2x) - 1$. Therefore, we only discuss the implementation of $\sigma(x)$ in the paper, since the implementation of $\text{Tanh}(x)$ is essentially the same as $\sigma(x)$. Based on the definitions of $\sigma(x)$ and $\text{Tanh}(x)$, these two computations can be securely executed by merging secure evaluations of secure e^x, reciprocal calculation, and multiplications. We begin by determining x's symbol $\text{sign}(x)$ and converting x to $|x|$ through the secure comparison with 0. In later computations, the input of the secure computation protocol is adjusted to $|x|$. This adjustment can lower communication overhead by providing initial values and reduce the iteration times in the reciprocal calculation. The final results can be obtained by

$$\sigma(x) = \begin{cases} \sigma(|x|) & \text{if } \text{sign}(x) \geq 0; \\ 1 - \sigma(|x|) & \text{if } \text{sign}(x) = -1. \end{cases} \tag{11}$$

The detailed construction of the above two protocols is given in Appendix B.7, available online.

6) Dropout Function: In the training process, the dropout operation randomly zeroes some of the input elements with probability p. Furthermore, the outputs are scaled by a factor of $\frac{1}{1-p}$. For convenience, we define the dropout function by

$$\text{Dropout}(x; p) = \begin{cases} 0, & \text{with probability } p; \\ \frac{x}{1-p}, & \text{else.} \end{cases} \tag{12}$$

We define the dropout gate $\mathcal{G}_{\text{drop}}$ as the family of functions $g_{\text{drop},l} : \mathbb{Z}_{2^l} \to \mathbb{Z}_{2^l}$ with input $\mathbb{G}^{\text{in}} = \mathbb{Z}_{2^l}$, output $\mathbb{G}^{\text{out}} = \mathbb{Z}_{2^l}$, and $g_{\text{drop},l}(x; p) := \text{Dropout}(x; p)$. We denote the dropout gate with FSS by $\hat{\mathcal{G}}_{\text{drop}}$ and the offset functions by:

$$\begin{aligned} \hat{g}_{\text{drop},l}^{[r^{\text{in}}, r^{\text{out}}]}(\hat{x}) &= g_{\text{drop},l}(\hat{x} - r^{\text{in}}) + r^{\text{out}} \mod 2^l \\ &= \frac{\hat{x} - r^{\text{in}}}{1 - p} \cdot \mathbf{1}\{r < p\} \end{aligned} \tag{13}$$

where $r \in (0, 1)$ is a random number. The straightforward way to implement a dropout gate is to generate random numbers belonging to the interval (0,1) during the offline stage and then apply secure comparison to compare these random numbers with p during the online stage. However, the key of the dropout function is to calculate $\sigma = \mathbf{1}\{r < p\}/(1 - p)$. Obviously, the comparison can be done during the offline stage, and the results of σ can be saved as keys to avoid the comparison operation during the online stage.

Based on the above analysis, we present the implementation of the dropout function with FSS in the Appendix B.8, available online. The proposed protocol reduces the communication overhead of the secure dropout in the online stage from $5ln$ bits down to $3ln$ bits compared to the implementation of CrypTen [11]. We refer to the above dropout method, which completes random number comparison in the offline stage, as the *static dropout*. A shortcoming is that the dropout probability must stay the same in secure federated fine-tuning. Therefore, we also consider the *dynamic dropout* in the evaluation, in which the key holds random values in the offline stage. The secure comparison calculates the dropout factor according to the random values and dynamic probability in the online stage.

7) Tensor Product of Vectors: We consider vectors $\mathbf{v} = \{v_1, v_2, \ldots, v_N\}$ and $\mathbf{w} = \{w_1, w_2, \ldots, w_M\}$ to discuss tensor product of vectors. The tensor product $\mathbf{v} \otimes \mathbf{w}$ generates a

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

$N \times M$ matrix such that

$$\mathbf{v} \otimes \mathbf{w} = \begin{pmatrix} v_1 w_1 & v_1 w_2 & \cdots & v_1 w_M \\ v_2 w_1 & v_2 w_2 & \cdots & v_2 w_M \\ \vdots & \vdots & \ddots & \vdots \\ v_N w_1 & v_N w_2 & \cdots & v_N w_M \end{pmatrix}. \quad (14)$$

The backpropagation of the model involves the tensor product of vectors, in which the linear layer gradient calculation involves the tensor product of the parameters and the gradients of the next layer. A straightforward way to implement the tensor product of vectors is to treat $\mathbf{v}$ and $\mathbf{w}$ as $n \times 1$ and $1 \times m$ matrices and obtain $\mathbf{v} \otimes \mathbf{w}$ through matrix multiplication.

Considering the training with batch size of B, the calculation of the gradient involves a $B \times N \times M$ tensor according to the tensor product of a $B \times N$ matrix and a $B \times M$ matrix. The above computation can be implemented in two ways. A simple method is to perform B matrix multiplications through a loop. However, this cannot load computing data to the GPU at one time, reducing computational efficiency. Another method is to extend both matrices to a tensor of $B \times M \times N$ and calculate the result by secure multiplication. The disadvantage is that the key size is $3BMNl$ bits, and the communication overhead is $5BMNl$ bits.

The tensor multiplication of vectors by matrix multiplication is more suitable for cases where the batch size is 1. However, it is hardly possible to set the batch size to 1 in reality. We implement the tensor product using FSS to apply GPU acceleration while reducing the key size and communication overhead. For ease of presentation, we still discuss the realization of the tensor product based on FSS through $\mathbf{v} \otimes \mathbf{w}$. The key generation of the tensor product in the offline phase is similar to that of the secure multiplication. The detailed implementations are given in Appendix B.9, available online. The innovation is that the implementation in PriFFT applies $\mathbf{r}^{\text{in}_1} \otimes \mathbf{r}^{\text{in}_2} \mod 2^l$ to generate an $N \times M$ matrix. In the online stage, the implementation in PriFFT calculates the result through the tensor product of the inputs with offsets and shares of masks. The detailed construction is given in Appendix B.9, available online.

D. Implementation and Example

Fig. 5 illustrates privacy-preserving LLMs implemented via hybrid secret sharing and optimized protocols based on FSS. The left side of the figure depicts an LLM with N decoders, adhering to the current mainstream Transformer architecture. When implementing a privacy-preserving LLM, PriFFT initially transform all specific computations in the model into corresponding secure operations using ASS. By leveraging multi-party secure computing libraries like CrypTen [11] that support machine learning, the fine-tuning process of LLMs can be transformed into corresponding secure computations based on ASS, thus avoiding the need to build a privacy-preserving fine-tuning framework from scratch. In the privacy-preserving fine-tuning of LLMs, various secure computation protocols process the input $(\langle x \rangle_0, \langle x \rangle_1)$ to generate the model secure output $(\langle y \rangle_0, \langle y \rangle_1)$.

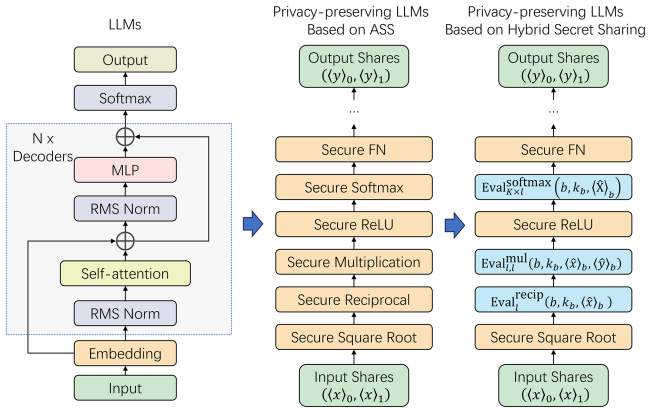


Fig. 5. An example of privacy-preserving LLMs based on PriFFT.

Current ASS-based secure computation protocols face performance limitations in LLM privacy-preserving processing. PriFFT boosts the efficiency of privacy-preserving fine-tuning with the optimized protocols based on FSS. Due to differences in input and output between FSS and ASS secure computation protocols, they cannot be directly combined. PriFFT employs hybrid secret sharing to replace the ASS-based bottleneck protocols in the fine-tuning of LLMs with optimized FSS-based protocols, thereby enhancing computational efficiency without increasing sharing conversion communication overhead.

Using the calculation of scaled dot-product attention in the Transformer architecture as an example, the following illustrates the specific implementation process of PriFFT. In plaintext computation, the scaled dot product attention can be obtained by $\text{Attention}(Q, K, V) = \text{Softmax}(\frac{QK^\top}{\sqrt{d_k}})V$. The vectors Q, K, and V are computed as $Q = W_Q x$, $K = W_K x$, and $V = W_V x$, where x is the input to the current decoder and W_Q, W_K, W_V are the corresponding weights. We assume that the ASS-based secure protocols $\text{Mul}(\langle x \rangle_b, \langle y \rangle_b)$ and $\text{Sfm}(\langle x \rangle_b)$ correspond to secure multiplication and Softmax function operations, respectively. The scaled dot product attention calculation based on ASS first calculates the query vector $\langle Q \rangle_b = \text{Mul}(\langle W_Q \rangle_b, \langle x \rangle_b)$, the key vector $\langle K \rangle_b = \text{Mul}(\langle W_K \rangle_b, \langle x \rangle_b)$, and the value vector $\langle V \rangle_b = \text{Mul}(\langle W_V \rangle_b, \langle x \rangle_b)$. Since the above $\sqrt{d_k}$ is not a sensitive parameter, the scaled dot product attention under the ciphertext can be calculated by the following formula:

$$\text{Mul}(\text{Sfm}(\frac{1}{\sqrt{d_k}} \cdot \text{Mul}(\langle Q \rangle_b, \langle Q \rangle_b^\top)), \langle V \rangle_b). \quad (15)$$

The above calculation can be optimized by replacing the corresponding secure calculation through the FSS-based protocols. Specifically, the secure computation with scaled dot-product attention first computes the query, key vector, and value vectors by

$$\begin{cases} \left\langle \hat{Q} \right\rangle_b = \text{Eval}_{l,l}^{\text{mul}}(b, k_b, \left\langle \hat{W}_Q \right\rangle_b, \langle \hat{x} \rangle_b); \\ \left\langle \hat{K} \right\rangle_b = \text{Eval}_{l,l}^{\text{mul}}(b, k_b, \left\langle \hat{W}_K \right\rangle_b, \langle \hat{x} \rangle_b); \\ \left\langle \hat{V} \right\rangle_b = \text{Eval}_{l,l}^{\text{mul}}(b, k_b, \left\langle \hat{W}_V \right\rangle_b, \langle \hat{x} \rangle_b). \end{cases} \quad (16)$$

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMMUNICATION OVERHEAD ANALYSIS OF THE PROPOSED PROTOCOLS IN THE ONLINE STAGE

Protocols	PriFFT	ABY2	CrypTen	SHAFT
$\text{Eval}_{K\times l}^{\text{softmax}}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$	$12l + 6lm_{\text{Softmax}}$	$16l + \log_2(K)\cdot 9l + 10lm_{\text{exp}} + 10lm_{\text{recip}}$		$10l + 10lm_{\text{Softmax}}$
$\text{Eval}_{l}^{\text{sigmoid}}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$	$20l + 3lm_{\text{exp}} + 6lm_{\text{recip}}$	$17l + 5lm_{\text{exp}} + 10lm_{\text{recip}}$		
$\text{Eval}_{l}^{\text{tanh}}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$				
$\text{Eval}_{l}^{\text{exp}}(b, k_b, \langle\hat{x}\rangle_b)$	$3l + 3lm_{\text{exp}}$	$l + 5lm_{\text{exp}}$		
$\text{Eval}_{l}^{\text{recip}}(b, k_b, \langle\hat{x}\rangle_b, y_0)$	$2l + 6lm_{\text{recip}}$	$10lm_{\text{recip}}$		
$\text{Eval}_{l}^{\text{recip}}(b, k_b, \langle\hat{x}\rangle_b)$	$5l + 3lm_{\text{exp}} + 6lm_{\text{recip}}$	$l + 5lm_{\text{exp}} + 10lm_{\text{recip}}$		
$\text{Eval}_{N\times l, M\times l}^{\text{TP}}(\mathbf{r}^{\text{in}_1}, \mathbf{r}^{\text{in}_2}, \mathbf{r}^{\text{out}})$	$2(M+N)l + MNl$	$5lMN$		
$\text{Eval}_{l}^{\text{power}}(b, k_b, \langle\hat{x}\rangle_b, 2^n)$	$3ln$	$5ln$		
$\text{Eval}_{l}^{\text{drop}}(b, k_b, \langle\hat{x}\rangle_b)$ (static)	$3l$	$5l$		
$\text{Eval}_{l}^{\text{drop}}(b, k_b, \langle\hat{x}\rangle_b)$ (dynamic)	$5l$	$6l$		

Since all the above secure multiplication operations involve $\hat{x}$, it is only necessary to restore $\hat{x}$ once during secure multiplication to complete the computation, avoiding multiple restoration to reduce communication overhead. The optimized scaled dot product attention is calculated as

$$\begin{cases} \langle\hat{t}\rangle_b = \text{Eval}_{l,l}^{\text{mul}}(b, k_b, \left\langle\hat{Q}\right\rangle_b, \left\langle\hat{K}\right\rangle_b^{\top}); \\ \langle\hat{t}\rangle_b = \text{Eval}_{K\times l}^{\text{sfotmax}}(b, k_b, \frac{1}{\sqrt{d_k}}\cdot\langle\hat{t}\rangle_b); \\ \langle\hat{y}\rangle_b = \text{Eval}_{l,l}^{\text{mul}}(b, k_b, \langle\hat{t}\rangle_b, \left\langle\hat{V}\right\rangle_b). \end{cases} \tag{17}$$

All ASS-based protocols in the above examples can be substituted with FSS-based protocols proposed in the paper. Fully achieving privacy-preserving fine-tuning of LLMs requires utilizing several ASS-based protocols, such as ReLU functions and Norm computations. Therefore, PriFFT employs hybrid secret sharing to seamlessly integrate the optimization protocol into existing multi-party secure computation libraries, thereby enabling privacy-preserving federated fine-tuning and reducing computational overhead.

VI. THEORETICAL ANALYSIS

A. Communication Overhead

The communication overhead of the proposed protocols is summarized in Table II. We compare the implementation of PriFFT, ABY2 [14], CrypTen [11], and SHAFT [12]. We next thoroughly evaluate the communication costs of each protocol in the online phase. Due to space limitations, we only discuss the communication cost of the proposed protocols in the paper.

$Eval_{K\times l}^{softmax}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$: The algorithm first performs ReLU on $(a_0 - \hat{\mathbf{x}}_b)$ and $(\hat{\mathbf{x}}_b - a_1)$ to clip the secure input. The communication cost of the ReLU function occurs in the secure multiplication of the input with its sign. Hence, the communication overhead for the ReLU operation above is $10l$ bits for two secure multiplications. Since $\hat{x}$ is used multiple times in each Iteration, it only needs to be restored once before the Iteration, with a communication cost of $2l$ bits. Each iteration requires restoring $\langle\mathbf{y}_{i-1}\rangle_b$ and $\sum\langle\mathbf{t}\rangle_b$ for secure multiplication, with a communication cost of $4lm_{\text{softmax}}$. Accounting for the $2lm_{\text{softmax}}$ bits truncation cost in secure multiplication, the total communication cost of $\text{Eval}_{K\times l}^{\text{softmax}}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$ is $12l + 6lm_{\text{softmax}}$ bits.

$Eval_{l}^{power}(b, k_b, \langle\hat{x}\rangle_b, 2^n)$: The algorithm calls n times $\text{Eval}_{l}^{\text{square}}(b, k_b^{\text{square}}, \langle\hat{t}_{i-1}\rangle_b)$. In each $\text{Eval}_{l}^{\text{square}}$ call, $\langle\hat{t}_{i-1}\rangle_b$ is restored, and a truncation is performed. Therefore, the communication cost of this algorithm is $3ln$ bits.

$Eval_{l}^{exp}(b, k_b, \langle\hat{x}\rangle_b)$: The algorithm restores $\hat{x}$ with $2l$ bits and calculates the initial value $(1 + x/2^{m_{\text{exp}}})$ through a truncation with a cost of l bits. The algorithm calls $\text{Eval}^{\text{square}} l$ m_{exp} times with the cost of $3lm_{\text{exp}}$ bits. The communication cost of this algorithm is $(3l + 3lm_{\text{exp}})$ bits.

$Eval_{l}^{recip}(b, k_b, \langle\hat{x}\rangle_b, y_0)$: When the input provides the initial iteration value y_0, the algorithm skips the call of secure e^x to compute the initial iteration value. The algorithm mainly includes restoring $\hat{x}$ ($2l$ bits), calling $\text{Eval}_{l}^{\text{square}}$ ($3lm_{\text{recip}}$ bits) and calling $\text{Eval}_{l,l}^{\text{mul}}$. Since $\hat{x}$ has been restored, the communication cost of $\text{Eval}_{l,l}^{\text{mul}}$ is $3lm_{\text{recip}}$ bits. Therefore, when y_0 is provided, the communication cost of this algorithm is $(2l + 6lm_{\text{recip}})$ bits.

$Eval_{l}^{recip}(b, k_b, \langle\hat{x}\rangle_b)$: The difference between this algorithm and $\text{Eval}_{l}^{\text{recip}}(b, k_b, \langle\hat{x}\rangle_b, y_0)$ is that the initial value is not given, and this algorithm needs to call secure e^x to calculate the initial value. Since the communication cost of $\text{Eval}_{l}^{\text{exp}}(b, k_b, \langle\hat{x}\rangle_b)$ is $(3 + 3lm_{\text{exp}})$ bits, the communication overhead of this algorithm is $(5l + 6lm_{\text{recip}} + 3lm_{\text{exp}})$ bits.

$Eval_{l}^{sigmoid}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$ and $Eval_{l}^{tanh}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$: As discussed in the paper, $\text{Eval}_{l}^{\text{tanh}}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$ can be implemented using $\text{Eval}_{l}^{\text{sigmoid}}(b, k_b, \langle\hat{\mathbf{x}}\rangle_b)$ with identical communication cost. The communication cost of the algorithm involves recovering $\hat{x}$ ($2l$ bits) and executing the multiplication of $\langle\hat{x}\rangle$ and $\langle\hat{t}\rangle$ ($3l$ bits). The algorithm subsequently invokes $\text{Eval}_{l}^{\text{exp}}$ and $\text{Eval}_{l}^{\text{recip}}$ with an initial value of 0.25, incurring communication costs of $(3l + 3lm_{\text{exp}})$ bits and $(2l + 6lm_{\text{recip}})$ bits, respectively. The algorithm ultimately performs two secure multiplications to compute the final result, with a communication cost of $10l$ bits. Therefore, the total communication overhead is $(20l + 3lm_{\text{exp}} + 6lm_{\text{recip}})$ bits.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

$Eval^{TP}_{N\times l,M\times l}(\mathbf{r}^{in_1},\mathbf{r}^{in_2},\mathbf{r}^{out})$: The algorithm restores $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ with the communication cost of $2Nl$ bits and $2Ml$ bits. Most of the remaining calculations can be completed by local calculations, except for the truncation. The truncation $\text{Trunc}(\langle\hat{t}\rangle_b)$ cost MNl bits. Therefore, the total communication overhead is $(2(N+M)l+NMl)$ bits.

$Eval^{drop}_l(b,k_b,\langle\hat{x}\rangle_b)$ (static) and $Eval^{drop}_l(b,k_b,\langle\hat{x}\rangle_b)$: For the static dropout function, the dropout factor has been saved in the key in the offline phase. Therefore, in this case, we only need to restore $\hat{x}$ and call the secure multiplication to complete the calculation with the communication cost of $3l$ bits. In dynamic dropout, the algorithm performs secure comparison through FSS and uses the result in secure multiplication. The comparison result is multiplied by the input $\hat{x}$, and then multiplied by the dropout factor to get the final result. Therefore, the communication overhead of the dynamic dropout is $5l$ bits.

B. Security Analysis

We prove the security of all proposed secure protocols against semi-honest attackers under the cryptographic standard of security definition [37] with the composition theorem [38]. Please refer to Appendix C (available online) for detailed security proofs.

VII. Evaluation

A. Evaluation Setup

Benchmark and datasets: We evaluate the LLMs fine-tuned by PriFFT on general language understanding evaluation (GLUE) [39]. We consider four classic GLUE training tasks in the evaluation: Stanford sentiment treebank (SST-2), Microsoft research paraphrase corpus (MRPC), recognizing textual entailment (RTE), and corpus of linguistic acceptability (CoLA). In evaluation, we compare various secure multi-party computation mechanisms ABY2 [14], CrypTen [11] and SHAFT [12]. ABY2 is an open-source library for secure multi-party computation based on arithmetic secret sharing. Similarly, CrypTen is a secure multi-party computing library which lso supports neural network inference and training. SHAFT optimizes some protocols based on CrypTen and realizes the secure inference of large models. The comparison mechanisms include the mainstream secure multi-party computation library (ABY2), the secure multi-party computation library designed for neural networks (CrypTen), and the state-of-the-art work in secure inference of large models (SHAFT).

Training models: We consider various language models in the evaluation: BERT (base and large) with 110 M and 340 M parameters [3]; RoBERTa (base and large) with 125 M and 355 M parameters [40]; DistilBERT with 67 M parameters [41]; ALBERT (base and large) with 12 M and 18 M parameters [42]; DeBERTa V1 (base and large) with 100 M and 350 M parameters [43]. We use the original public versions of these models, meaning none have been fine-tuned for specific downstream tasks. The backbone of the models is frozen, and we fine-tune the poolers and classifiers according to the training tasks.

TABLE III
Overhead of Secure Softmax(x)

Input Size		10^2	10^3	10^4	10^5	10^6
Time (ms)	PriFFT	105.4	107.1	107.8	137.0	472.8
	ABY2	334.0	382.4	435.4	515.0	910.6
	CrypTen	337.1	376.5	452.1	501.0	912.3
	SHAFT	131.7	133.9	134.3	171.8	637.6
Comm (MB)	PriFFT	0.08	0.82	8.24	82.40	823.97
	ABY2	0.19	2.12	23.96	260.66	2807.62
	CrypTen	0.19	2.12	23.96	260.66	2807.62
	SHAFT	0.13	1.30	12.97	129.70	1297.00

Metrics: For SST-2, MRPC, and RTE, we consider inference accuracy (Acc) as the metric. The metric for CoLA is the Matthews correlation coefficient (MCC). MCC is a metric to measure the performance of classification models, especially in cases of category imbalance, providing a value between -1 and 1, where 1 indicates perfect classification, 0 indicates random classification, and -1 indicates complete misclassification. Besides, we also focus on the communication overhead and execution time of fine-tuning LLMs and each protocol proposed in this paper. Communication overhead includes the traffic of sending data and receiving data, while execution time includes local execution time and data transmission time.

Experiment environment: All experiments are conducted on a system equipped with an NVIDIA RTX 4090 GPU with 24 GB of memory and an Intel Xeon Gold 6430 CPU. We evaluate our privacy-preserving mechanisms and protocols in the LAN setting with 0.21 ms round-trip latency and 2.5 Gbps network bandwidth. PriFFT is implemented by PyTorch and NssMPClib[1], while all computations are accelerated by CUDA. Evaluation works in the secret sharing domain, where all data are represented as integers and shared over the ring $\mathbb{Z}_{2^{64}}$. The scale factor s is set to 16. The number of clients in FL is set to 20.

B. Protocol Analysis

This section analyzes the execution time and communication overhead of protocols mentioned in the paper under various mechanisms. In SHAFT and CrypTen, the standard network setting is configured to utilize local computing resources. To maintain experimental consistency, the execution time reported in this section is measured within the identical local computing environment. More specifically, we emulate multiple parties using multiple processes on an NVIDIA RTX 4090, facilitating data exchange via shared memory for communication. Furthermore, we evaluate the execution times of various protocols within a LAN setting.

Table III displays the overhead associated with various implementations of the secure Softmax function. We optimize the implementation of the secure softmax function within PriFFT, enabling PriFFT to realize the lowest execution time and lowest

[1]https://github.com/XidianNSS/NssMPClib

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EXECUTION TIME AND COMMUNICATION OVERHEAD OF SECURE $1/x$

		Without initial values					With initial values				
Input Size		10^3	10^4	10^5	10^6	10^7	10^3	10^4	10^5	10^6	10^7
Time (ms)	PriFFT	74.0	74.2	80.1	200.4	1876.5	54.1	54.3	59.0	132.8	1207.9
	ABY2	80.2	82.7	94.0	232.5	2440.1	65.8	72.9	96.4	197.6	1683.9
	CrypTen	110.5	110.7	130.1	429.3	4017.7	96.8	102.7	114.8	356.7	3222.9
	SHAFT	100.5	109.1	140.3	433.8	4023.1	93.1	101.5	109.8	358.5	3276.0
Comm (MB)	PriFFT	0.74	7.40	74.01	740.05	7400.51	0.55	5.49	54.93	549.32	5493.16
	ABY2	1.15	11.52	115.20	1152.04	11520.4	0.83	8.39	83.92	839.23	8392.33
	CrypTen	1.15	11.52	115.20	1152.04	11520.4	0.83	8.39	83.92	839.23	8392.33
	SHAFT	1.15	11.52	115.20	1152.04	11520.4	0.83	8.39	83.92	839.23	8392.33

communication overhead for the secure softmax function compared to other mechanisms. The clipping range is set to $(-4, 12)$, and $m_{\text{Softmax}} = 16$, matching the evaluation configuration in SHAFT. SHAFT provides an approximation of the softmax function, bypassing the need to determine the maximum value in the input vector. This method reduces communication overhead and decreases execution time when compared to both ABY2 and CrypTen. Since ABY2 does not optimize the softmax function, the softmax function in ABY2 relies on the foundational operations of ABY2 combined with the implementation strategy of the softmax in CrypTen. Consequently, both ABY2 and CrypTen incur identical communication costs when applying the softmax function. Compared to SHAFT, PriFFT decreases execution time by approximately 25% and communication overhead by around 36% in the secure softmax function.

Table IV illustrates the overhead of secure $1/x$ in different mechanisms. Similar to the secure Softmax(x), the methodology of $1/x$ in ABY2 is the same as the logic employed by CrypTen, with the exception that $1/x$ is reconstructed using ABY2's foundational protocol. Therefore, ABY2, CrypTen, and SHAFT have the same communication consumption, and have a minor difference in execution time. PriFFT enhances secure $1/x$ operations by optimizing input mask size in the secure multiplication, thereby decreasing communication overhead and eliminating some steps of shared data restoration to achieve reduced execution time.

As discussed in Section V-C3, the accuracy of the reciprocal operation is related to initial values. In the absence of a specified initial value, the secure protocol computes $(3e^{1-2x} + 0.003)$ as the default, leading to extra overhead. Providing approximate initial values when the input range is known can not only eliminate calculating initial values, but also decrease the number of iterations, thereby minimizing computational cost. Table IV presents the overhead of the secure reciprocal operation, both with and without initial values. The difference is nearly equivalent to a call to the secure e^x. Compared to SHAFT, PriFFT decreases execution time by approximately 63% and communication overhead by around 35% in the secure $1/x$ protocol.

Tables V and VI illustrate the overhead of the secure e^x and Sigmoid(x) protocols. In the isolated assessment of the secure e^x protocol's overhead, m_{exp} is configured to 8, aligning with CrypTen's default parameters. Compared to SHAFT, PriFFT reduces 53% execution time and 20% communication overhead in the secure e^x protocol. The secure sigmoid protocol exhibits a moderate level of complexity, given that it requires secure zero comparisons, $1/x$, and e^x. We maintain the sigmoid parameters identical to the default configuration in CrypTen. Specifically, m_{recip} in the secure reciprocal is set to 3 with an initial value of 0.75. The sigmoid function produces outputs in the range (0,1), allowing the protocol to decrease the number of iterations and thereby minimize computational overhead. Compared to SHAFT, PriFFT decreases execution time by approximately 8% and communication overhead by around 33% in the secure Sigmoid(x) protocol.

TABLE V
OVERHEAD OF SECURE e^x

Input Size		10^3	10^4	10^5	10^6	10^7
Time (ms)	PriFFT	16.0	16.2	18.2	46.8	478.7
	ABY2	33.3	35.3	37.5	71.4	543.7
	CrypTen	32.9	35.2	36.5	78.2	542.5
	SHAFT	34.1	37.0	38.3	75.3	547.8
Comm (MB)	PriFFT	0.19	1.91	19.07	190.73	1907.35
	ABY2	0.31	3.13	31.28	312.81	3128.05
	CrypTen	0.31	3.13	31.28	312.81	3128.05
	SHAFT	0.31	3.13	31.28	312.81	3128.05

TABLE VI
OVERHEAD OF SECURE SIGMOID(x)

Input Size		10^3	10^4	10^5	10^6	10^7
Time (ms)	PriFFT	63.5	68.0	77.9	292.5	2677.0
	ABY2	93.3	94.5	110.7	342.8	3016.3
	CrypTen	87.9	94.0	106.3	314.6	2795.9
	SHAFT	87.6	92.5	104.4	317.41	2736.9
Comm (MB)	PriFFT	0.55	5.49	54.93	549.32	5493.16
	ABY2	0.76	7.63	76.29	762.94	7629.39
	CrypTen	0.76	7.63	76.29	762.94	7629.39
	SHAFT	0.76	7.63	76.29	762.94	7629.39

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
FINE-TUNED MODELS PERFORMANCE COMPARISON UNDER VARIOUS DATASETS AND SETTINGS. PRIFFT WITH THE ITERATIVE AND LOCAL TRUNCATION (PRIFFT-IT AND PRIFFT-LT) CAUSE SLIGHT PERFORMANCE LOSS COMPARED TO THE FINE-TUNING WITH PLAINTEXT (PT)

Tasks	SST-2 (Acc)			MRPC (Acc)			RTE (Acc)			CoLA (MCC)		
	PT	PriFFT-IT	PriFFT-LT	PT	PriFFT-IT	PriFFT-LT	PT	PriFFT-IT	PriFFT-LT	PT	PriFFT-IT	PriFFT-LT
BERT-base	85.13%	84.21%	83.72%	73.77%	72.16%	70.58%	61.01%	58.85%	57.43%	0.441	0.428	0.420
BERT-large	86.76%	84.79%	83.23%	75.74%	73.23%	72.92%	62.09%	60.93%	60.11%	0.461	0.449	0.432
RoBERTa-base	83.71%	81.93%	81.02%	70.83%	69.01%	67.91%	54.87%	53.01%	51.94%	0.386	0.370	0.357
RoBERTa-large	85.55%	84.01%	82.96%	71.57%	69.86%	68.51%	57.76%	56.32%	54.79%	0.404	0.389	0.372
DistilBERT	84.28%	82.45%	81.79%	72.06%	70.48%	69.98%	58.84%	57.15%	55.64%	0.358	0.342	0.331
ALBERT-base	83.83%	82.35%	81.78%	76.96%	76.13%	74.85%	67.51%	66.23%	65.11%	0.380	0.369	0.361
ALBERT-large	85.24%	83.89%	82.51%	77.70%	76.84%	76.01%	68.73%	67.91%	66.86%	0.441	0.435	0.429
DeBERTa-base	82.90%	82.11%	80.83%	70.30%	68.17%	66.95%	60.65%	58.73%	58.02%	0.499	0.487	0.481
DeBERTa-large	85.56%	84.78%	83.62%	71.08%	69.53%	68.47%	63.34%	61.98%	61.26%	0.520	0.505	0.482

C. Performance and Overhead of Fine-Tuning LLMs

We evaluate the performance of fine-tuned models with different settings, then discuss the training process's overhead in the LAN setting. PriFFT is the first mechanism that presents privacy-preserving federated fine-tuning to protect model parameters and updates based on SMPC. Therefore, we implement an ABY2-based mechanism to realize the fine-tuning of LLMs for comparison. Table VII summarizes the best performance of fine-tuned models under various tasks, where PT, PriFFT-IT, and PriFFT-LT refer to the training implemented in plaintext, iterative truncations, and local truncations, respectively. PriFFT-IT applies iterative truncations for all multiplications. Matrix multiplication in PriFFT-LT is implemented via iterative truncations to improve the accuracy, and protocols proposed in Section V are implemented via local truncations to reduce the overhead. In addition to the GLUE performance experiments, we also test the performance of LLMs on the CNN/daily mail summarization task [44]. The experiments are conducted using fine-tuned BART [45], PEGASUS [46], and DistilBART. The experiment calculates the ROUGE score of the model in the summary task under privacy-preserving inference [47]. The experimental results are given in Appendix D, available online.

Fig. 6 illustrates how the accuracy on SST-2 of the fine-tuned model (BERT-base) varies with the aggregation times. There is little difference between ciphertext and plaintext fine-tuning before the model converges, and models fine-tuned with plaintext can achieve higher accuracy. The calculation error caused by the fine-tuning on encrypted samples does not bring significant model performance degradation for three reasons. Firstly, extensive training on the tunable parts of the model parameters can offset the impact of computational errors to a certain extent during the fine-tuning. Secondly, gradient values determine the change in model parameters, and gradients in the optimization step are generated by a batch of training samples. Each parameter update of the model is the average of multiple calculation results, reducing the impact of a single calculation error. Finally, model training often introduces randomness into the calculations to prevent overfitting. The success of introducing randomness in model training shows that model training is not sensitive to small amounts of noise.

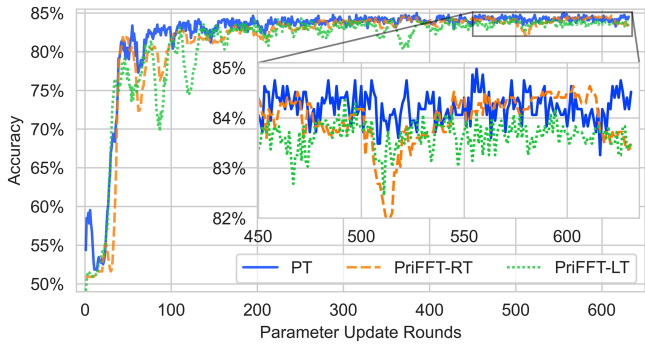


Fig. 6. The relationship between the accuracy of BERT on SST-2 and parameter update rounds.

TABLE VIII
COMMUNICATION CONSUMPTION OF PRIVACY-PRESERVING FINE-TUNING FOR BERT WITH A BATCH OF SAMPLES

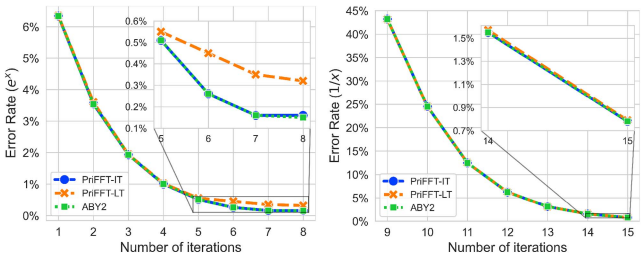
Batch Size		8	16	32	64
BERT-base	PriFFT-IT	61.43MB	104.78MB	191.86MB	364.9MB
	PriFFT-LT	25.27MB	28.46MB	39.21MB	59.60MB
	ABY2	210.05MB	402.06MB	786.06MB	1.46GB
BERT-large	PriFFT-IT	105.90MB	179.70MB	327.29MB	622.5MB
	PriFFT-LT	39.02MB	44.94MB	59.77MB	87.44MB
	ABY2	368.01MB	701.06MB	1.34GB	2.66GB

We next discuss the communication and computational consumption during privacy-preserving fine-tuning in the LAN setting. Table VIII summarizes the communication overhead when fine-tuning BERT with a batch of samples under different privacy-preserving implementations. Table IX summarizes the execution time of corresponding privacy-preserving fine-tuning.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
EXECUTION TIME OF PRIVACY-PRESERVING FINE-TUNING FOR BERT WITH A BATCH OF SAMPLES

Batch Size			8	16	32	64
CPU	BERT-base	PriFFT-IT	0.83s	1.34s	2.42s	4.64s
		PriFFT-LT	0.67s	0.72s	1.17s	1.75s
		ABY2	1.04s	1.85s	4.28s	7.60s
	BERT-large	PriFFT-IT	1.52s	2.81s	4.53s	9.07s
		PriFFT-LT	0.74s	1.13s	1.95s	3.78s
		ABY2	1.53s	3.39s	6.80s	11.52s
GPU	BERT-base	PriFFT-IT	0.78s	1.01s	1.41s	2.98s
		PriFFT-LT	0.51s	0.55s	0.58s	1.23s
		ABY2	1.12s	1.41s	3.15s	6.73s
	BERT-large	PriFFT-IT	1.05s	1.36s	1.88s	3.12s
		PriFFT-LT	0.64s	0.68s	0.73s	0.99s
		ABY2	1.48s	1.86s	5.29s	10.51s



(a) e^x (b) $1/x$

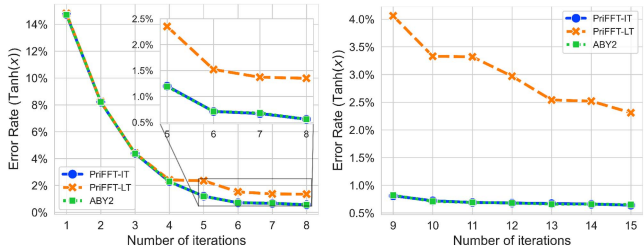
Fig. 7. The impact of the number of iterations on the error rates of the secure e^x and reciprocal calculation.

We also compare the communication consumption and execution time of privacy-preserving fine-tuning of BERT through all training samples of downstream tasks. Due to the space limitation, the results are given in Appendix D, available online. In addition, we also test the communication cost of Qwen3 [48] with different parameter sizes in privacy-preserving fine-tuning for downstream tasks. The experimental results are given in Appendix D, available online.

D. Error Evaluation

PriFFT approximates e^x, $1/x$, and $\text{Tanh}(x)$ by the secure computation. Meanwhile, we provide different truncation methods in the paper. In this section, we combine evaluation to discuss the errors of each secure calculation under different settings. Evaluation first calculates the relative error between the approximated results from the secure computation and the exact results computed in plaintext with PyTorch. We use the error rate to analyze the error in the secure computation, i.e., the percentage between the relative errors and the exact values. The inputs are 10^5 random values from 0 to 1, and the provided results are the mean of 10^5 error rates for each case.

Fig. 7 presents the impact of the number of iterations m_{exp} and m_{recip} on the error rates of secure e^x and $1/x$. For secure e^x, when $m_{\text{exp}} < 5$, truncation methods have no significant impact on the error rate, as shown in Fig. 7(a). When the number of



(a) e^x (b) $1/x$

Fig. 8. The impact of m_{exp} and m_{recip} on the error rates of the secure $\text{Tanh}(x)$.

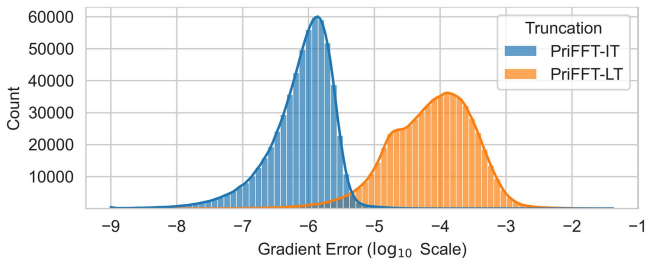


Fig. 9. The distribution of gradient error.

iterations is insufficient, the error is relatively large. In this case, the difference between iterative and local truncations is too small to impact the error rate significantly. The error rate of secure e^x decreases as m_{exp} increases. Meanwhile, differences in truncation methods begin to have an impact on error. Experimental results show that it is difficult to effectively reduce the error rates by further increasing the number of iterations. It comes from that the experiment takes 16 bits to represent the fractional part of fixed-point numbers. More accurate results rely on more bits to represent the fractional part.

Fig. 7(b) presents the error rate of the secure reciprocal calculation under different m_{recip}. Despite applying a higher number of iterations, the secure reciprocal calculation does not have a lower error rate than the secure e^x. The reason is that different approximation calculations were performed during the iteration process. As shown in (4), setting the number of iterations to 8 for the secure e^x implies that $n = 2^8 = 256$ for $(1 + x/n)^n$. However, the number of iterations equals the number of approximations for the secure reciprocal calculation, leading the secure e^x to realize lower error rates with fewer iterations.

Fig. 8 provides the impact of m_{exp} and m_{recip} on the error rates of $\text{Tanh}(x)$. The implementation of the secure $\text{Tanh}(x)$ involves the secure e^x and the reciprocal calculation. Fig. 8(a) presents the error rate of the secure $\text{Tanh}(x)$ under different m_{exp} when $m_{\text{recip}} = 15$. Fig. 8(a) presents the error rate of the secure $\text{Tanh}(x)$ under different m_{recip} when $m_{\text{exp}} = 8$. Although the error rate of the secure reciprocal calculation is higher than that of the secure e^x, the secure reciprocal calculation has a minor effect on the error rate of the secure $\text{Tanh}(x)$ compared to the secure e^x.

We further analyze the impact of different truncation methods on the model gradient error. Fig. 9 compares the error of weight gradients in the pooler layer generated by 32 samples with

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

different truncation methods. PriFFT-IT achieves smaller errors, owing to its higher accuracy in secure computation. However, most of the errors in PriFFT-LT lie between 10^{-3} and 10^{-6}, which is still minor compared to the gradients. Therefore, it is more practical to employ LT to reduce communication consumption during the early stages of fine-tuning and subsequently apply IT to enhance calculation accuracy in the later stages of fine-tuning.

VIII. Conclusion

In this paper, we first discuss the privacy concern in federated fine-tuning, i.e., both LLMs' parameters and uploaded gradients would cause privacy leakage. To solve the above privacy problem, we present PriFFT to implement federated fine-tuning of LLMs, while protecting both model parameters and uploaded gradients. In PriFFT, clients and the server additively share LLM parameters and the inference results, while the privacy-preserving fine-tuning is implemented based on the shared values. Each party cannot directly access the plaintext LLM parameters and clients' gradients in the privacy-preserving fine-tuning. Due to the large amount of LLM parameters, fine-tuning LLMs on shared values requires substantial computation and communication resources. Therefore, we propose several FSS-based secure protocols and the hybrid secret sharing combining ASS and FSS to implement the privacy-preserving federated fine-tuning and reduce the overhead. We provide detailed overhead and secure analysis for PriFFT. Besides, we evaluate PriFFT with privacy-preserving federated fine-tuning implemented by existing mechanisms. Evaluation results show that PriFFT realizes the same fine-tuned model accuracy while significantly reducing the communication overhead and the execution time.

References

[1] J. OpenAI et al., "GPT-4 technical report," 2024, *arXiv:2303.08774*.

[2] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2019, *arXiv:1810.04805*.

[4] N. Carlini et al., "Extracting training data from large language models," 2021, *arXiv:2012.07805*.

[5] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2017, *arXiv:1610.05492*.

[6] C. Zhang, T. Huang, W. Mao, H. Bai, and B. Yu, "Federated continual learning based on central memory rehearsal," *Hospital*, vol. 1, 2024, Art. no. T2.

[7] H. Xiao, X. Mu, and K. Cheng, "FedRMA: A robust federated learning resistant to multiple poisoning attacks," *J. Netw. Netw. Appl.*, vol. 4, no. 1, pp. 31–38, 2024.

[8] C. Chen, X. Feng, J. Zhou, J. Yin, and X. Zheng, "Federated large language model: A position paper," 2023, *arXiv:2409.15723*.

[9] Z. You, X. Dong, S. Li, X. Liu, S. Ma, and Y. Shen, "Local differential privacy is not enough: A sample reconstruction attack against federated learning with local differential privacy," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1519–1534, 2025.

[10] X. Yin, Y. Zhu, and J. Hu, "A comprehensive survey of privacy-preserving federated learning: A taxonomy, review, and future directions," *ACM Comput. Surv.*, vol. 54, no. 6, Jul. 2021, doi: 10.1145/3460427.

[11] B. Knott, S. Venkataraman, A. Y. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, "Crypten: Secure multi-party computation meets machine learning," 2021, *arXiv:2109.00984*.

[12] A. Y. Kei and S. S. Chow, "SHAFT: Secure, handy, accurate, and fast transformer inference," in *Proc. Int. Conf. Netw. Distrib. System Secur. Symp.*, 2025, pp. 1024–1037.

[13] K. Gupta, D. Kumaraswamy, N. Chandran, and D. Gupta, "LLAMA: A low latency math library for secure inference," *Cryptol. ePrint Arch.*, Paper 2022/793, 2022. [Online]. Available: https://eprint.iacr.org/2022/793

[14] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "{ABY2. 0}: Improved {Mixed-Protocol } secure { Two-Party} computation," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2165–2182.

[15] Z. Qin, D. Chen, B. Qian, B. Ding, Y. Li, and S. Deng, "Federated full-parameter tuning of billion-sized language models with communication cost under 18 kilobytes," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024. [Online]. Available: https://openreview.net/forum?id=cit0hg4sEz

[16] D. Cai, Y. Wu, S. Wang, and M. Xu, "Fedadapter: Efficient federated learning for mobile NLP," in *Proc. ACM Turing Award Celebration Conf. China*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 27–28, doi: 10.1145/3603165.3607380.

[17] Z. Zhang et al., "FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Findings of the Association for Computational Linguistics: ACL 2023*. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 9963–9977. [Online]. Available: https://aclanthology.org/2023.findings-acl.632

[18] H. Zhao, W. Du, F. Li, P. Li, and G. Liu, "FedPrompt: Communication-efficient and privacy-preserving prompt tuning in federated learning," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2023, pp. 1–5.

[19] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3454–3469, 2020.

[20] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. 2020 USENIX Annu. Tech. Conf.*, 2020, pp. 493–506.

[21] T. Eltaras, F. Sabry, W. Labda, K. Alzoubi, and Q. Ahmedeltaras, "Efficient verifiable protocol for privacy-preserving aggregation in federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2977–2990, 2023.

[22] H. Fereidooni et al., "Safelearn: Secure aggregation for private federated learning," in *Proc. 2021 IEEE Secur. Privacy Workshops*, 2021, pp. 56–62.

[23] A. Yazdinejad, A. Dehghantanha, H. Karimipour, G. Srivastava, and R. M. Parizi, "A robust privacy-preserving federated learning model against model poisoning attacks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6693–6708, 2024.

[24] A. Song, J. Fu, X. Mu, X. Zhu, and K. Cheng, "L-secNet: Towards secure and lightweight deep neural network inference," *J. Netw. Netw. Appl.*, vol. 3, no. 4, pp. 171–181, 2024.

[25] C. Guo et al., "GFS-CNN: A GPU-friendly secure computation platform for convolutional neural networks," *J. Netw. Netw. Appl.*, vol. 3, no. 2, pp. 66–72, 2023.

[26] S. Kadhe, N. Rajaraman, O. O. Koyluoglu, and K. Ramchandran, "FastSecAgg: Scalable secure aggregation for privacy-preserving federated learning," 2020, *arXiv:2009.11248*.

[27] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "ELSA: Secure aggregation for federated learning with malicious actors," *Cryptol. ePrint Arch.*, Paper 2022/1695, 2022. [Online]. Available: https://eprint.iacr.org/2022/1695

[28] C. Hahn, H. Kim, M. Kim, and J. Hur, "Versa: Verifiable secure aggregation for cross-device federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 36–52, Jan./Feb. 2023.

[29] W. Huang, Y. Wang, A. Cheng, A. Zhou, C. Yu, and L. Wang, "A fast, performant, secure distributed training framework for large language model," 2024, *arXiv:2401.09796*.

[30] A. Muñoz, R. Ríos, R. Román, and J. López, "A survey on the (in)security of trusted execution environments," *Comput. Secur.*, vol. 129, 2023, Art. no. 103180. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404823000901

[31] E. Boyle, N. Gilboa, and Y. Ishai, "Secure computation with preprocessing via function secret sharing," in *Proc. 17th Int. Conf. Theory Cryptogr.*, Nuremberg, Germany, 2019, pp. 341–371.

[32] J. Fu, K. Cheng, A. Song, Y. Xia, Z. Chang, and Y. Shen, "FSS-DBSCAN: Outsourced private density-based clustering via function secret sharing," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7759–7773, 2024.

[33] S. Wang, Y. Zheng, X. Jia, and C. Wang, "Ggrass: An encrypted attributed subgraph matching system with malicious security," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5999–6014, 2024.

[34] J. Fu, K. Cheng, Y. Xia, A. Song, Q. Li, and Y. Shen, "Private decision tree evaluation with malicious security via function secret sharing," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2024, pp. 310–330.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.

[35] X. Mu et al., "FedDMC: Efficient and robust federated learning via detecting malicious clients," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5259–5274, Nov./Dec. 2024.
[36] X. Mu, K. Cheng, T. Liu, T. Zhang, X. Geng, and Y. Shen, "FedPTA: Prior-based tensor approximation for detecting malicious clients in federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9100–9114, 2024.
[37] Y. Lindell, "How to simulate it–a tutorial on the simulation proof technique," in *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich*. Berlin, Germany: Springer, 2017, pp. 277–346.
[38] R. Canetti, "Security and composition of multiparty cryptographic protocols," *J. CRYPTO.*, vol. 13, pp. 143–202, 2000.
[39] A. Wang, "Glue: A multi-task benchmark and analysis platform for natural language understanding," 2018, *arXiv:1804.07461*.
[40] Y. Liu et al., "Roberta: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
[41] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
[42] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," 2019, *arXiv:1909.11942*.
[43] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," 2020, *arXiv:2006.03654*.
[44] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," 2017, *arXiv:1704.04368*.
[45] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019, *arXiv:1910.13461*.
[46] J. Zhang, Y. Zhao, M. Saleh, and P. J. Liu, "PEGASUS: Pre-training with extracted gap-sentences for abstractive summarization," 2019, *arXiv:1912.08777*.
[47] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/
[48] A. Yang et al., "Qwen3 technical report," 2025, *arXiv:2505.09388*.
[49] E. Boyle, N. Gilboa, and Y. Ishai, "Function secret sharing," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2015, pp. 337–367.



Zhichao You received the BE degree in information and computing science from South China Agricultural University, Guangzhou, China in 2018, the MS degree in computer science and technology from Xidian University, Xi'an, China, in 2021. He is currently working toward the PhD degree in computer science and technology with Xidian University, Xi'an, China. His research interests include privacy-preserving federated learning and wireless network security.



Xuewen Dong (Member, IEEE) received the BE, MS, and PhD degrees in computer science and technology from Xidian University, Xi'an, China, in 2003, 2006, and 2011, respectively. From 2016 to 2017, he was with Oklahoma State University, OK, USA, as a visiting scholar. Now, he is an associate professor with the School of Computer Science, Xidian University. His research interests include cognitive radio networks, wireless network security, and blockchain.



Ke Cheng received the PhD degree in computer science and technology from Xidian University, Xi'an, Shaanxi, China, in 2022. He is an associate professor with the School of Computer Science and Technology, Xidian University. His research interests include data security and privacy protection.



Xutong Mu received the BS degree from the North University of China, China, in 2019, and the PhD degree in computer science and technology from Xidian University, in 2024. His research interests include machine learning security and federated learning.



Jiaxuan Fu received the BE and PhD degrees in computer science and technology from Xidian University, Xi'an, China, in 2019 and 2025, respectively. His research interests include the IoT data security and machine learning security.



Shiyang Ma received the BE degree in telecommunications engineering from Xidian University, Xi'an, China, in 2015, and is currently working toward the PhD degree with the school of Computer Science and Technology, Xidian University. His current research interests include semantic technology, service computing, IoT, and blockchain.



Qiang Qu received the PhD degree from Aarhus University, Aarhus, Denmark, supported by the GEOCrowd project under Marie Skodowska-Curie Actions. He is now a full professor with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences. His current research interests include large-scale data management and mining, blockchain, and data intelligence.



Yulong Shen (Senior Member, IEEE) received the BS and MS degrees in computer science and the PhD degree in cryptography from Xidian University, Xi'an, China, in 2002, 2005, and 2008, respectively. He is currently a professor with the School of Computer Science and Technology, Xidian University, and also an associate director of the Shaanxi Key Laboratory of Network and System Security. His research interests include wireless network security and cloud computing security.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 20:01:44 UTC from IEEE Xplore. Restrictions apply.